Scraped data

After data is scraped from different sources, it can be persisted into a file using *FeedExports*, which ensures that data is stored properly with multiple serialisation formats. We will store the data as XML. Run the following command to store the data:

```
scrapy crawl myFirstSpider -o data.xml
```

We can find *data.xml* in our project's root folder, as shown in Figure 3.

Figure 3: *data.xml*

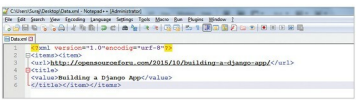

Figure 4: Final spider

Our final spider will look like what's shown in Figure 4.

Built-in services

1. *Logging:* Scrapy uses Python's built-in logging system for event tracking. It allows us to include our own messages along with third party APIs' logging messages in our application's log.

```
import logging
logging.WARNING('this is a warning')
logging.log(logging.WARNING,"Warning Message")
logging.error("error goes here")
logging.critical("critical message goes here")
logging.info("info goes here")
logging.debug("debug goes here")
```

2. *stats collection:* This facilitates the collection of stats in a key value pair, where values are often *counter*. This service is always available even if it is disabled, in which case the API will be called but will not collect anything.
   The stats collector can be accessed using the stats attribute. For example:

```
class ExtensionThatAccessStats(object):
    def __init__(self,stats):
      self.stats=stats
    @classmethod
    def from_crawler(cls,crawler):
      return cls(crawlse.stats)
Set stat value: "stats.set_value('hostname',socket.
gethostname())"
Increment stat value: "stats.inc_value('count_variable')"
Get stat value: "stats.get_stats()"
```

3. *Sending email:* Scrapy comes with an easy-to-use service for sending email and is implemented using a twisted non-blocking IO of the crawler. For example:

```
    from scrapy.mail import MailSender
    mailer=MailSender()
mailer.send(to=['abc@xyz.com'],subject="Test Subject "
,body="Test Body", cc=['cc@abc.com'])
```

4. Telnet console: All the running processes of Scrapy are controlled and inspected using this console. It comes enabled by default and can be accessed using the following command:

```
telnet console 6023
```

5. Web services: This service is used to control Scrapy's Web crawler via the JSON-RPC 2.0 protocol. It needs to be installed separately using the following command:

```
pip install scrapy-jsonrpc
```

The following lines should be included in our project's *settings.py* file:

```
EXTENSIONS={'scrapy_jsonrpc.webservice.WebService':500,}
Set JSONRPC_ENABLED settings to True.
```

Scrapy vs BeautifulSoup

***Scrapy*:** Scrapy is a full-fledged spider library, capable of performing load balancing restrictions, and parsing a wide range of data types with minimal customisation. It is a Web scraping framework and can be used to crawl numerous URLs by providing constraints. It is best suited in situations like when you have proper seed URLs. Scrapy supports both CSS selectors and XPath expressions for data extraction. In fact, you could even use BeautifulSoup or PyQuery as the data extraction mechanism in your Scrapy spiders.

BeautifulSoup: This is a parsing library which provides easy-to-understand methods for navigating, searching and finally extracting the data you need, i.e., it helps us to navigate through HTML and can be used to fetch data and parse it into any specific format. It can be used if you'd rather implement the HTML fetching part yourself and want to easily navigate through HTML DOM. END

Reference

https://doc.scrapy.org

By: Shubham Sharma

The author is an open source activist working as a software engineer at KPIT Technologies, Pune. He can be contacted at *shubham.ks494@gmail.com*.